Title: Book Table of Contents Using a Tree

1. Problem Statement

To develop a C++ program that stores a book's table of contents using a tree data structure. The tree represents chapters, sections, and subsections in a hierarchical structure. The program displays the table of contents with proper indentation and counts the total number of leaf sections.

2. Objectives

- To understand the concept of tree data structures.
- To represent chapters, sections, and subsections hierarchically.
- To display the table of contents with proper indentation.
- To identify and count leaf nodes in a tree.
- To develop a practical application of tree data structures using C++.

4. Introduction

A tree is a non-linear data structure used to represent hierarchical relationships between elements. It consists of nodes connected by edges.

In a book, the table of contents follows a hierarchical structure:

- Root: The main book.
- Children: Chapters of the book.
- Subchildren: Sections within chapters.
- Subsections: Smaller topics within sections.
- Leaf nodes: Nodes that do not have any children.

A tree is suitable for representing a book's table of contents because it organizes information in a structured manner.

This project uses a tree to store the table of contents, print it with indentation, and count the leaf nodes.

5. Theory

5.1 Tree Data Structure

A tree is a collection of nodes connected through edges. It begins with a root node and branches into child nodes.

5.2 Important Terminology

Term

Meaning

Node

An individual element in a tree

Root

The topmost node

Parent

A node that has one or more children

Child

A node connected below another node

Edge

A connection between two nodes

Leaf Node

A node that has no children

Subtree

A smaller tree within a tree

Depth

The number of edges from the root to a node

5.3 Application of Trees

Trees are used in:

Book table of contents

File and folder systems

Organization charts

HTML document structure

Database indexing

Decision-making systems

5.4 Leaf Node Counting

A leaf node is a node that does not have any children.

6. Algorithm

Step 1: Start the program.

Step 2: Define a tree node containing a title and a list of child nodes.

Step 3: Create the root node representing the book.

Step 4: Add chapters as children of the root node.

Step 5: Add sections as children of the corresponding chapters.

Step 6: Add subsections under their respective sections.

Step 7: Display the tree recursively with indentation according to the node's depth.

Step 8: Check each node:

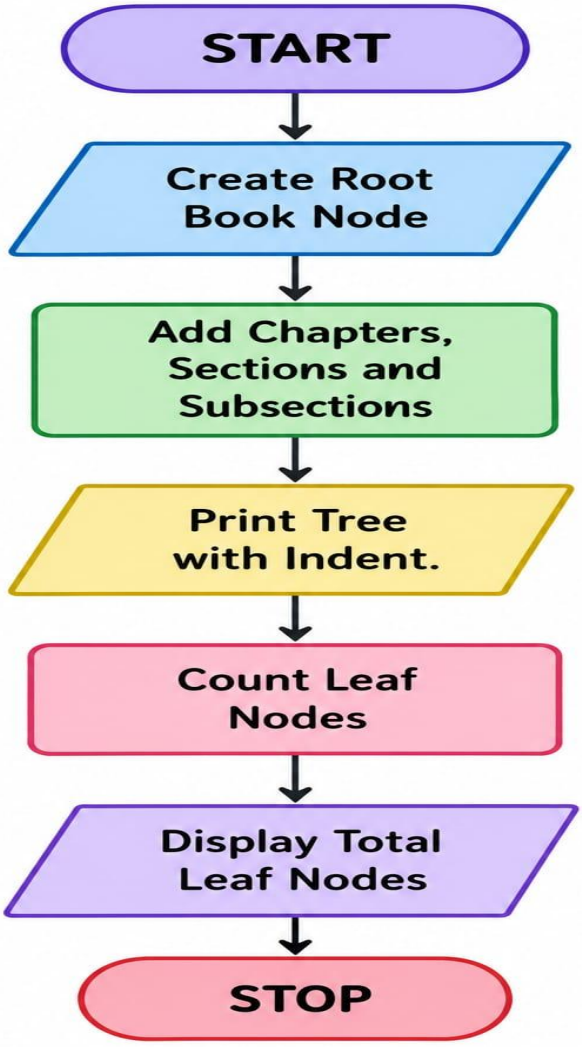
If the node has no children, increase the leaf count by one.

Otherwise, recursively count the leaf nodes of its children.

Step 9: Display the total number of leaf nodes.

Step 10: Stop the program.

7. Flowchart



8. Advantages

- Organizes information hierarchically.
- Makes the table of contents easy to read.
- Supports multiple chapters and sections.
- Counts leaf nodes automatically.
- Demonstrates recursion and dynamic memory allocation.
- Can be extended to accept user input.

9. Limitations

- The current program uses a predefined table of contents.
- It does not save the tree permanently to a file.
- It does not provide editing or deletion options for individual sections.
- Very deep trees may cause excessive recursive function calls.

10. Conclusion

This project successfully demonstrates how a tree data structure can represent a book's table of contents in a hierarchical format. The C++ program displays chapters, sections, and subsections with proper indentation using recursion. It also counts all leaf nodes, including sections and subsections that have no children. Thus, the project provides a practical understanding of trees, recursion, and node traversal in data structures.